

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvores Avançadas

Relatório técnico - Trie, Patricia, Splay, Treap e KD-Tree

Aluno: _____

Disciplina: Algoritmos e Estruturas de Dados II

Professor(a): _____

Data: 19 de setembro de 2026

Resumo. Este trabalho apresenta a implementação e a avaliação experimental de cinco estruturas de dados hierárquicas especializadas: Trie, árvore Patricia, árvore Splay, Treap e KD-Tree. As implementações foram desenvolvidas em C++17 e contemplam inserção, busca, remoção, liberação de memória e visualização estrutural, além de operações específicas quando pertinentes. Os experimentos empregaram conjuntos com 500, 1000, 2500 e 5000 elementos, sob padrões de entrada distintos, e mediram os tempos de inserção, busca e remoção. Os resultados evidenciam diferenças relevantes entre estruturas orientadas a strings, árvores de busca autoajustáveis ou probabilísticas e estruturas espaciais, reforçando que a escolha adequada depende do tipo de dado e do padrão de acesso.

1. Introdução e Fundamentação Teórica

Estruturas em árvore organizam informações por relações hierárquicas e são fundamentais para busca, indexação, ordenação e representação espacial. Uma árvore binária de busca (BST) preserva uma ordem entre chaves, mas pode se degenerar em uma lista encadeada e atingir custo $O(n)$ por operação. A AVL reduz esse problema impondo balanceamento explícito e garante altura $O(\log n)$, ao custo de armazenar informação de balanceamento e executar rotações após atualizações [1]. As estruturas estudadas neste trabalho adotam estratégias diferentes: exploração de prefixos, compactação, autoajuste, aleatorização e particionamento multidimensional.

A Trie representa strings pelo compartilhamento de prefixos. Cada caractere conduz a um novo estado da busca e, por isso, o custo das operações depende principalmente do comprimento L da chave, e não diretamente do número n de palavras armazenadas. O conceito foi formalizado nos trabalhos clássicos sobre busca por chaves de comprimento variável e “trie memory” [2]. É especialmente apropriada para dicionários, autocompletar e consultas por prefixo.

A Patricia (Practical Algorithm To Retrieve Information Coded in Alphanumeric) é uma forma compactada de árvore de prefixos. Caminhos sem bifurcação são combinados em um único nó que armazena um segmento da chave, reduzindo a quantidade de nós e o número de transições necessárias. Morrison apresentou a estrutura como uma alternativa econômica em espaço de índice e flexível para recuperação de chaves alfanuméricas [3].

A árvore Splay é uma BST autoajustável. Ela não mantém fator de balanceamento; em vez disso, após um acesso, aplica rotações para mover o elemento consultado para a raiz. Sleator e Tarjan demonstraram custo amortizado $O(\log n)$ para as operações usuais, embora uma operação individual possa custar $O(n)$ [4]. O comportamento favorece sequências com localidade temporal, pois elementos recentemente acessados tendem a permanecer próximos da raiz.

A Treap combina duas invariantes: ordem de BST pela chave e propriedade de heap pela prioridade. Quando as prioridades são independentes e aleatórias, a estrutura produz balanceamento probabilístico e custo esperado $O(\log n)$ para busca, inserção e remoção, mantendo pior caso $O(n)$ [5]. Na implementação desenvolvida foi utilizado max-heap por prioridade.

A KD-Tree organiza pontos em k dimensões alternando a dimensão de comparação ao longo dos níveis. A implementação deste trabalho utiliza $k = 2$: profundidades pares comparam a coordenada X e profundidades ímpares comparam Y . A estrutura foi introduzida por Bentley para buscas associativas multidimensionais [6]. Quando o particionamento é razoavelmente equilibrado, operações de busca e inserção tendem a apresentar comportamento logarítmico; distribuições desfavoráveis podem degenerar a árvore.

2. Projeto e Implementação das Estruturas

O projeto foi organizado com interfaces em `lib/`, implementações em `src/`, testes e benchmark em `tests/`, script de gráficos em `scripts/` e resultados em `resultados/`. Todas as estruturas realizam alocação dinâmica de nós e possuem destrutores que percorrem a árvore para liberar a memória restante. Chaves duplicadas ou pontos duplicados são ignorados nas implementações em que essa política é aplicável.

Estrutura	Anatomia do nó	Decisão de projeto
Trie	<code>fimPalavra</code> ; <code>unordered_map<char, Node*></code> filhos	Um nó por caractere; marca de término separa palavra completa de mero prefixo.
Patricia	<code>prefixo</code> ; <code>fimPalavra</code> ; <code>unordered_map<char, Node*></code> filhos	Cada nó guarda um segmento de string e pode ser dividido/compactado.
Treap	<code>chave</code> ; <code>prioridade</code> ; <code>esquerda</code> ; <code>direita</code>	BST pela chave e max-heap pela prioridade.
Splay	<code>chave</code> ; <code>esquerda</code> ; <code>direita</code> ; <code>pai</code>	Ponteiro para pai facilita as rotações Zig/Zag e casos duplos.
KD-Tree	<code>Ponto{x,y}</code> ; <code>esquerda</code> ; <code>direita</code>	O eixo de comparação é definido por profundidade % 2.

Tabela 1 - Representação computacional dos nós nas implementações.

Na Trie, a inserção percorre cada caractere e cria filhos ausentes. A busca usa `find()` no `unordered_map` para não modificar a estrutura durante consultas. A remoção é recursiva: ao retornar das chamadas, um nó somente é apagado quando deixa de representar fim de palavra e não possui filhos. A função `palavrasPrefixo` localiza o nó correspondente ao prefixo e coleta recursivamente todas as palavras abaixo dele.

A Patricia utiliza a função `prefixoComum` para identificar quantos caracteres coincidem entre a parte restante da nova chave e o segmento já armazenado. Quando a coincidência é parcial, um nó intermediário é criado, o prefixo antigo é dividido e os dois sufixos passam a constituir ramificações distintas. Na remoção, nós sem utilidade são eliminados e nós com um único filho podem ser novamente compactados.

Na Treap, a inserção começa como em uma BST. Se a prioridade do filho recém-inserido ultrapassa a prioridade do pai, uma rotação restaura a propriedade de max-heap. A versão normal gera prioridades com `std::mt19937`; uma sobrecarga permite prioridade explícita para testes determinísticos. A remoção empurra o nó-alvo para baixo por rotações, promovendo o filho de maior prioridade, até que o nó possa ser eliminado.

A Splay usa ponteiros para pai e seis configurações de reorganização: Zig, Zag, Zig-Zig, Zag-Zag, Zig-Zag e Zag-Zig. Inserções e buscas bem-sucedidas chamam `splay()`, levando o nó acessado à raiz. Na remoção, o alvo é primeiro levado à raiz; as subárvores são separadas e o maior elemento da subárvore esquerda é promovido antes da reconexão com a subárvore direita.

Na KD-Tree, inserir e buscar recebem a profundidade da chamada para selecionar o eixo corrente. Na remoção, o substituto é escolhido pelo menor valor no eixo considerado. A comparação exata de `double` é suficiente para o benchmark, que reutiliza as mesmas coordenadas inseridas.

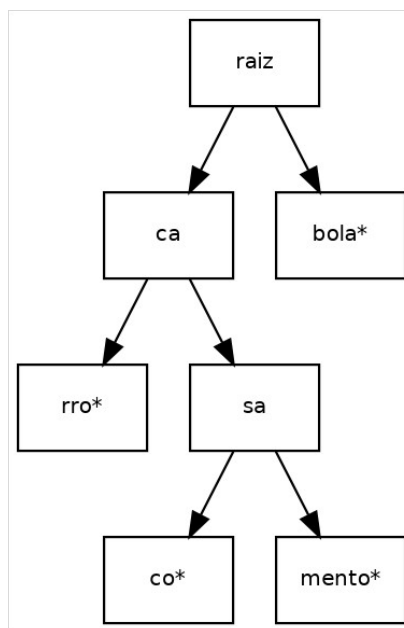
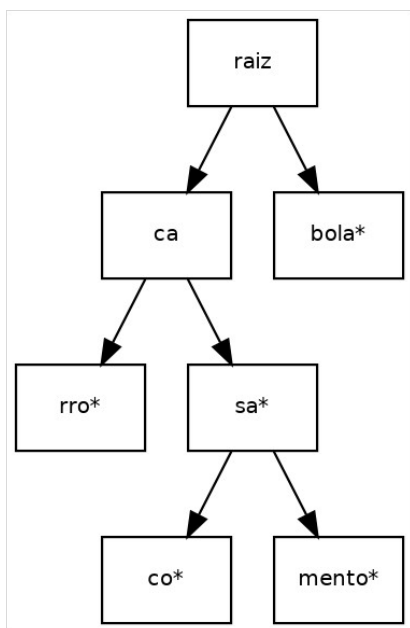


Figura 2 - Patricia: O segmento “sa” deixa de marcar fim de palavra, mas continua necessário para “casaco” e “casamento”.

Treap

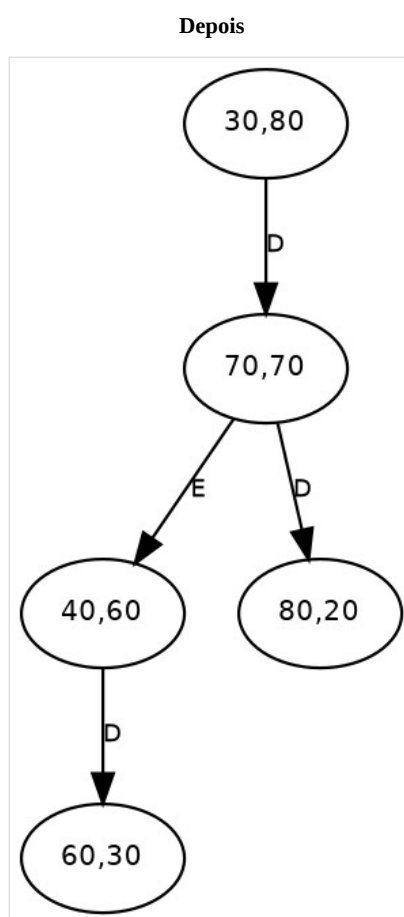
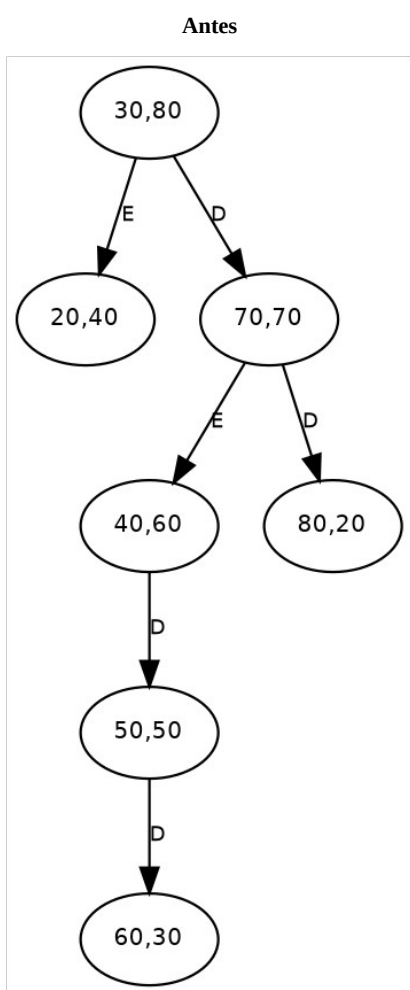


Figura 3 - Treap: A remoção preserva simultaneamente a ordenação pelas chaves e a propriedade de max-heap das prioridades.

Splay

Antes

Depois

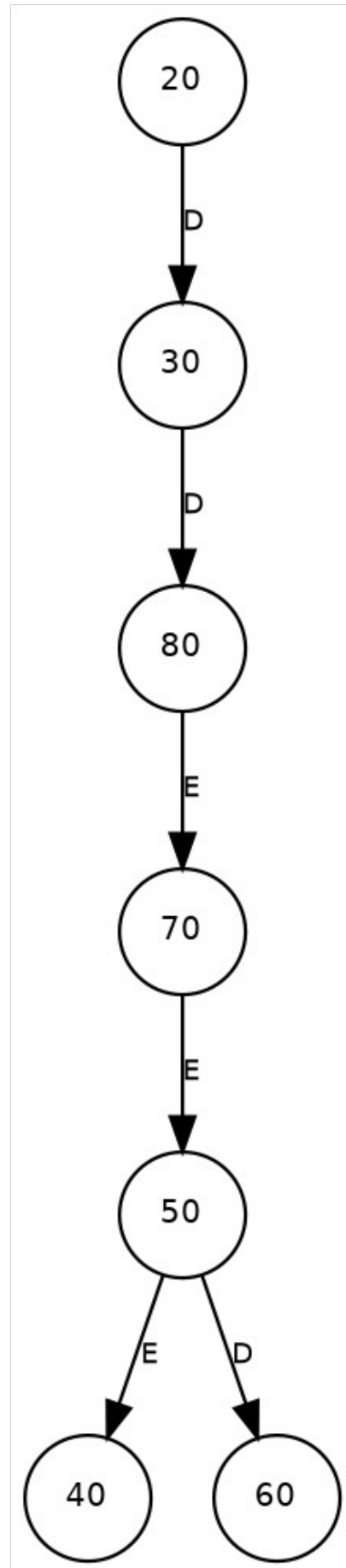
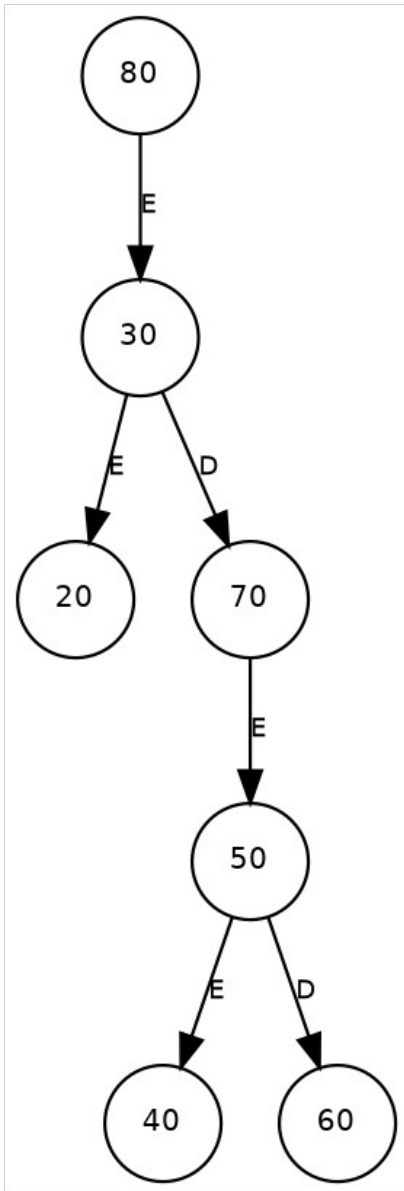


Figura 4 - Splay: Após a busca por 20, rotações sucessivas levam o elemento acessado até a raiz.

KD-Tree

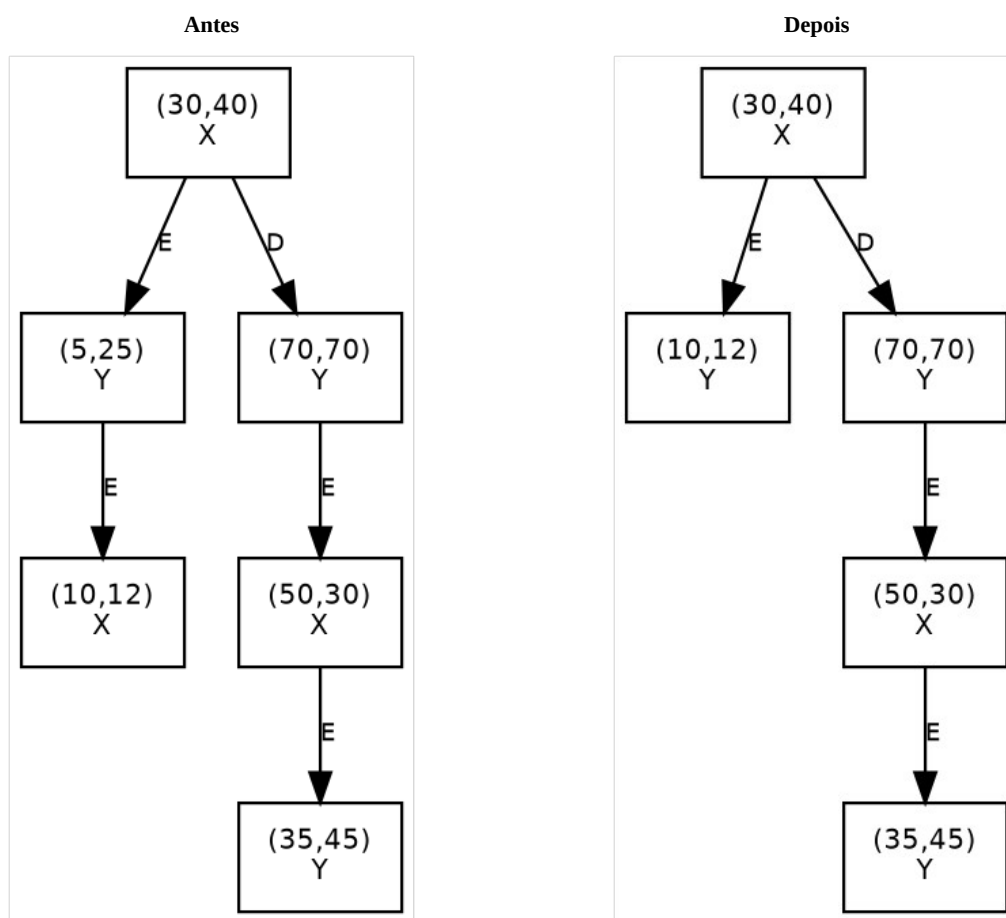


Figura 5 - KD-Tree: Após remover (5,25), o particionamento continua alternando os eixos X e Y de acordo com a profundidade.

4. Análise de Complexidade e Comparação Teórica

Na tabela seguinte, n representa a quantidade de chaves/pontos e L o comprimento da chave textual. Os custos de Trie e Patricia são expressos em L porque essas estruturas percorrem caracteres ou segmentos da chave. Para Treap, “esperado” pressupõe prioridades independentes e aleatórias. Para Splay, o limite $O(\log n)$ é amortizado, e uma operação isolada pode custar $O(n)$. Na KD-Tree, os custos médios pressupõem particionamento razoavelmente equilibrado; o pior caso ocorre quando a árvore se degenera.

Estrutura	Busca	Inserção	Remoção	Observação
BST	$O(\log n)$ médio; $O(n)$ pior	$O(\log n)$ médio; $O(n)$ pior	$O(\log n)$ médio; $O(n)$ pior	Pode degenerar sem balanceamento.
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Altura garantidamente logarítmica; usa rotações e fator de balanceamento.
Trie	$O(L)$	$O(L)$	$O(L)$	Espaço proporcional ao total de caracteres/nós.
Patricia	$O(L)$	$O(L)$	$O(L)$	Compacta caminhos sem bifurcação; reduz número de nós.
Treap	$O(\log n)$ esperado; $O(n)$ pior	$O(\log n)$ esperado; $O(n)$ pior	$O(\log n)$ esperado; $O(n)$ pior	Balanceamento probabilístico por prioridades.
Splay	$O(\log n)$ amort.; $O(n)$	$O(\log n)$ amort.; $O(n)$	$O(\log n)$ amort.; $O(n)$	Autoajuste favorece

	isolado	isolado	isolado	localidade de acesso.
KD-Tree 2D	$O(\log n)$ médio; $O(n)$ pior	$O(\log n)$ médio; $O(n)$ pior	$O(\log n)$ médio para nó aleatório; $O(n)$ pior	Desempenho depende fortemente da distribuição espacial.

Tabela 2 - Comparação assintótica das estruturas estudadas com BST e AVL.

A comparação evidencia que AVL fornece garantia de pior caso logarítmico para chaves ordenáveis, enquanto Treap e Splay buscam eficiência por mecanismos menos rígidos: aleatorização e autoajuste, respectivamente. Trie e Patricia mudam a unidade de análise: o custo está ligado à extensão da string. A KD-Tree, por sua vez, representa partições do espaço e sua qualidade depende da geometria e da ordem dos pontos.

5. Metodologia Experimental e Resultados

O benchmark utiliza `std::chrono::steady_clock` e executa cada cenário três vezes, registrando a média. Foram usados tamanhos $n = 500, 1000, 2500$ e 5000 . Para cada estrutura, inserção foi medida em uma instância vazia; busca e remoção foram medidas após pré-carregar a estrutura. O CSV registra tempo total em microssegundos e tempo médio por operação. Todas as estruturas foram avaliadas no mesmo ambiente de execução e o alvo de benchmark foi compilado com otimização `-O2`. Os tempos são, portanto, úteis para comparação relativa neste experimento, não como valores universais de desempenho.

Os dados textuais foram divididos em palavras variadas, formadas por oito letras pseudoaleatórias acrescidas de um identificador, e palavras com prefixo compartilhado no formato “estrutura_dados_i”. Para Treap e Splay foram utilizadas chaves inteiras ordenadas e uma permutação pseudoaleatória; na Treap, as prioridades também foram geradas de forma reprodutível. Para a KD-Tree foram usados pontos pseudoaleatórios e o caso desfavorável (i,i), no qual todos os pontos se encontram sobre uma diagonal.

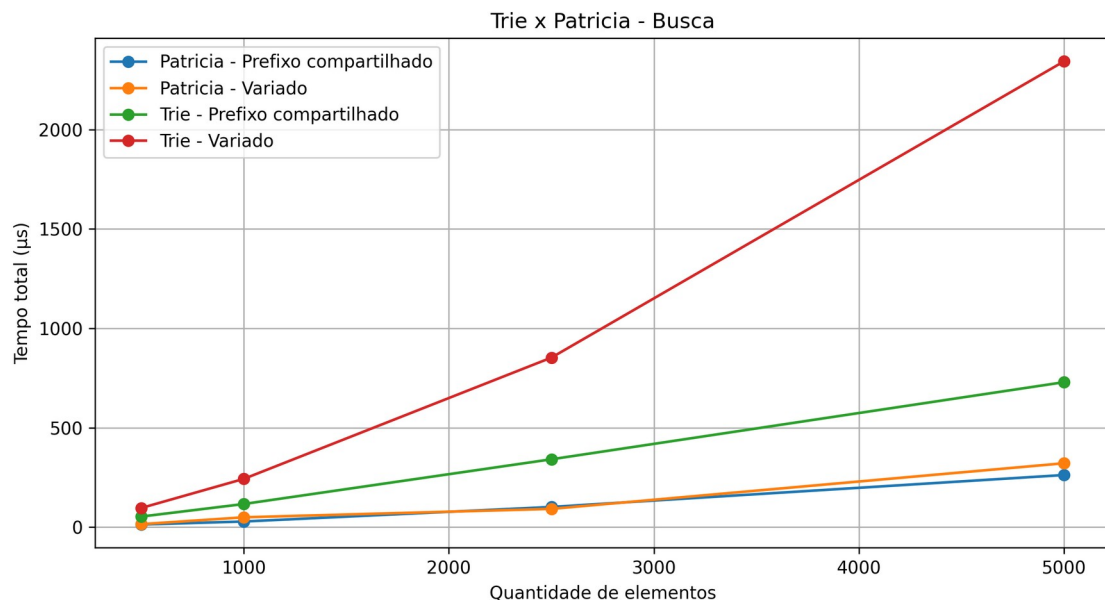


Figura 6 - Busca em Trie e Patricia para os dois padrões de strings.

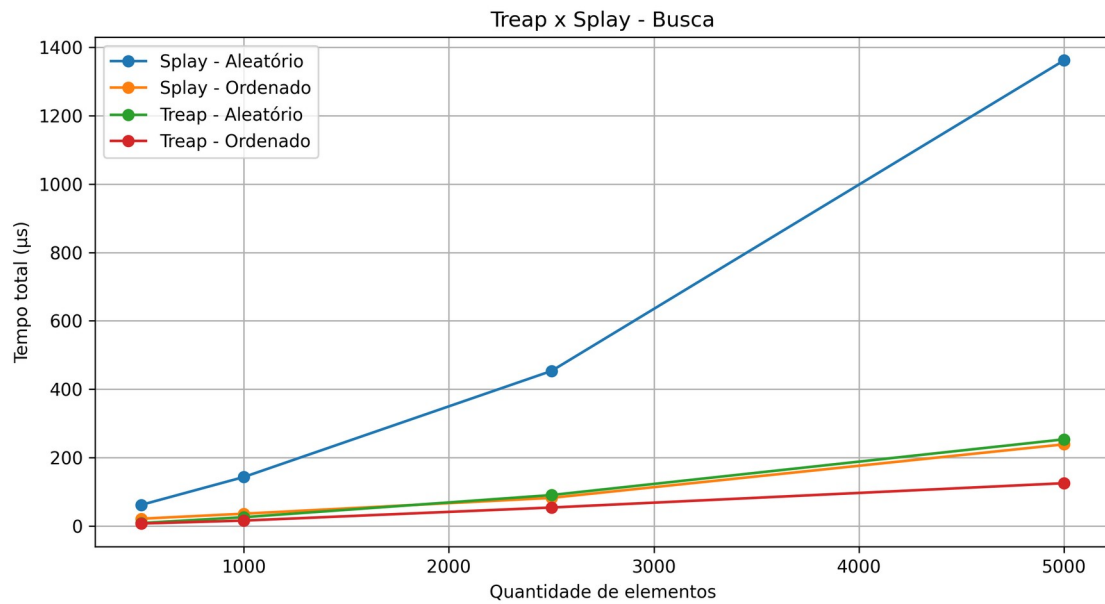


Figura 7 - Busca em Treap e Splay para chaves ordenadas e aleatórias.

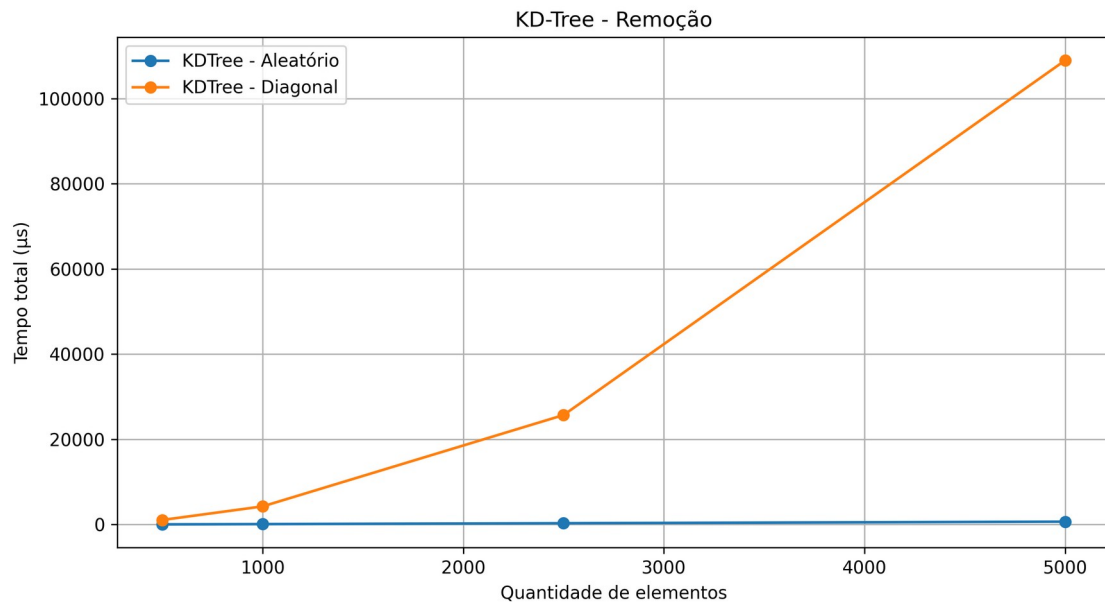


Figura 8 - Remoção na KD-Tree: contraste entre pontos aleatórios e distribuição diagonal.

Estrutura	Padrão	Inserção (ms)	Busca (ms)	Remoção (ms)
Trie	Variado	6.876	2.343	5.645
Trie	PrefixoCompartilhado	1.591	0.729	1.436
Patricia	Variado	0.876	0.322	0.791
Patricia	PrefixoCompartilhado	0.547	0.263	0.512
Treap	Ordenado	0.145	0.125	0.150
Treap	Aleatorio	0.569	0.253	0.473
Splay	Ordenado	0.067	0.239	0.161
Splay	Aleatorio	1.043	1.362	1.106
KDTree	Aleatorio	0.470	0.409	0.673
KDTree	Diagonal	66.865	53.494	108.970

Tabela 3 - Tempos médios das três repetições para $n = 5000$. Valores em milissegundos.

Nos experimentos de strings com $n = 5000$, a Patricia apresentou tempos menores que a Trie nos dois padrões. No conjunto variado, a diferença foi de aproximadamente 7.9 vezes na inserção, 7.3 vezes na busca e 7.1 vezes na remoção. No conjunto com prefixo compartilhado, as diferenças foram menores, mas ainda ficaram próximas de 2.9, 2.8 e 2.8 vezes, respectivamente. O resultado é compatível com a redução de nós e transições obtida pela compactação da Patricia. Também evidencia um custo prático relevante da Trie implementada com um `unordered_map` por nó, sobretudo em palavras variadas com pouco compartilhamento.

Entre Treap e Splay, o comportamento dependeu fortemente do padrão. Com chaves aleatórias e $n = 5000$, a Splay consumiu cerca de 1.8 vezes o tempo da Treap na inserção, 5.4 vezes na busca e 2.3 vezes na remoção. Já a sequência ordenada favoreceu a inserção da Splay, pois cada nova chave máxima pode ser promovida por uma reorganização simples. As buscas do benchmark também percorrem as chaves na ordem do conjunto, o que cria uma sequência com forte localidade. Assim, os resultados não devem ser interpretados como superioridade universal de entradas ordenadas, mas como consequência do padrão específico de acessos e do autoajuste da Splay.

A KD-Tree mostrou a maior sensibilidade à distribuição dos dados. Em $n = 5000$, o padrão diagonal foi aproximadamente 142 vezes mais lento na inserção, 131 vezes na busca e 162 vezes na remoção que o padrão aleatório. O caso (i,i) faz as decisões sucessivas por X e Y apontarem frequentemente para o mesmo lado, aproximando a estrutura de uma árvore degenerada. O comportamento experimental, portanto, acompanha a diferença entre o caso médio e o pior caso previstos teoricamente.

6. Aplicações, Análise Crítica e Discussão

A Trie é adequada quando consultas por prefixo são parte central do problema, como autocompletar, dicionários e roteamento por cadeias. Sua principal vantagem é a previsibilidade do percurso por caractere; sua limitação é o consumo de memória quando existem muitos nós esparsos. A implementação com `unordered_map` torna o alfabeto flexível, mas aumenta a sobrecarga por nó em relação a vetores fixos.

A Patricia é apropriada para indexação compacta e conjuntos de strings nos quais longos trechos não apresentam bifurcação. A redução estrutural foi refletida nos tempos experimentais, sobretudo no conjunto variado. Em contrapartida, inserir e remover exige lógica adicional de divisão e fusão de segmentos, tornando a implementação mais suscetível a erros do que a Trie convencional.

A Splay é útil quando acessos recentes ou frequentes tendem a se repetir. Ela dispensa metadados de altura ou cor, porém cada acesso pode alterar substancialmente a forma da árvore, o que dificulta prever o custo de uma operação individual. A Treap oferece implementação relativamente simples de balanceamento probabilístico e mantém busca compatível com BST; entretanto, suas garantias são esperadas, não absolutas, e dependem da qualidade/independência das prioridades.

A KD-Tree é apropriada para pontos multidimensionais, indexação espacial, GIS e problemas geométricos. O experimento diagonal evidencia sua principal limitação: sem construção balanceada ou reconstrução, distribuições adversas podem degradar fortemente o desempenho. Uma extensão natural seria implementar construção balanceada por mediana e operações como busca por região ou vizinho mais próximo.

Como limitações do experimento, não foi medido consumo de memória, os conjuntos possuem no máximo 5000 elementos e cada cenário foi repetido três vezes. Além disso, Trie/Patricia, Treap/Splay e KD-Tree recebem tipos de dados diferentes; por isso, os gráficos foram separados por domínio. Uma comparação absoluta entre todas as cinco estruturas pelo menor tempo seria metodologicamente inadequada.

7. Conclusão

As cinco estruturas resolvem problemas distintos e evidenciam estratégias diferentes de organização hierárquica. Trie e Patricia exploram prefixos de strings; a Patricia compacta caminhos e, nesta implementação, apresentou menor tempo nas três operações medidas. Treap e Splay preservam a semântica de uma BST, mas atingem eficiência por mecanismos diferentes: prioridades aleatórias na Treap e reorganização após acesso na Splay. A KD-Tree utiliza a árvore para particionar um espaço multidimensional e mostrou dependência especialmente forte da distribuição dos pontos.

Os resultados experimentais foram coerentes com as propriedades teóricas em aspectos importantes. A compactação da Patricia reduziu o trabalho observado em relação à Trie; a Splay respondeu de modo distinto a sequências com alta localidade e a acessos aleatórios; e a KD-Tree aproximou-se do pior caso no conjunto diagonal. A Treap apresentou comportamento estável e compatível com balanceamento probabilístico. Assim, o desempenho não pode ser separado do padrão dos dados e das operações realizadas.

Quanto à dificuldade de implementação, a Trie foi a estrutura mais direta, enquanto Patricia e KD-Tree exigiram maior cuidado na remoção e reorganização. A Splay demandou atualização consistente dos ponteiros de pai durante rotações, e a Treap exigiu manter simultaneamente duas invariantes. Em um sistema real, a escolha deve considerar o domínio: prefixos e autocompletar favorecem Trie/Patricia; acessos com localidade favorecem Splay; busca ordenada com balanceamento probabilístico favorece Treap; e consultas espaciais justificam KD-Tree. Não há uma única estrutura superior para todos os cenários.

Referências

- [1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.
- [2] FREDKIN, E. Trie memory. Communications of the ACM, v. 3, n. 9, p. 490-499, 1960. DOI: 10.1145/367390.367400.
- [3] MORRISON, D. R. PATRICIA - Practical Algorithm To Retrieve Information Coded in Alphanumeric. Journal of the ACM, v. 15, n. 4, p. 514-534, 1968. DOI: 10.1145/321479.321481.
- [4] SLEATOR, D. D.; TARJAN, R. E. Self-Adjusting Binary Search Trees. Journal of the ACM, v. 32, n. 3, p. 652-686, 1985. DOI: 10.1145/3828.3835.
- [5] SEIDEL, R.; ARAGON, C. R. Randomized Search Trees. Algorithmica, v. 16, n. 4-5, p. 464-497, 1996. DOI: 10.1007/BF01940876.
- [6] BENTLEY, J. L. Multidimensional Binary Search Trees Used for Associative Searching. Communications of the ACM, v. 18, n. 9, p. 509-517, 1975. DOI: 10.1145/361002.361007.